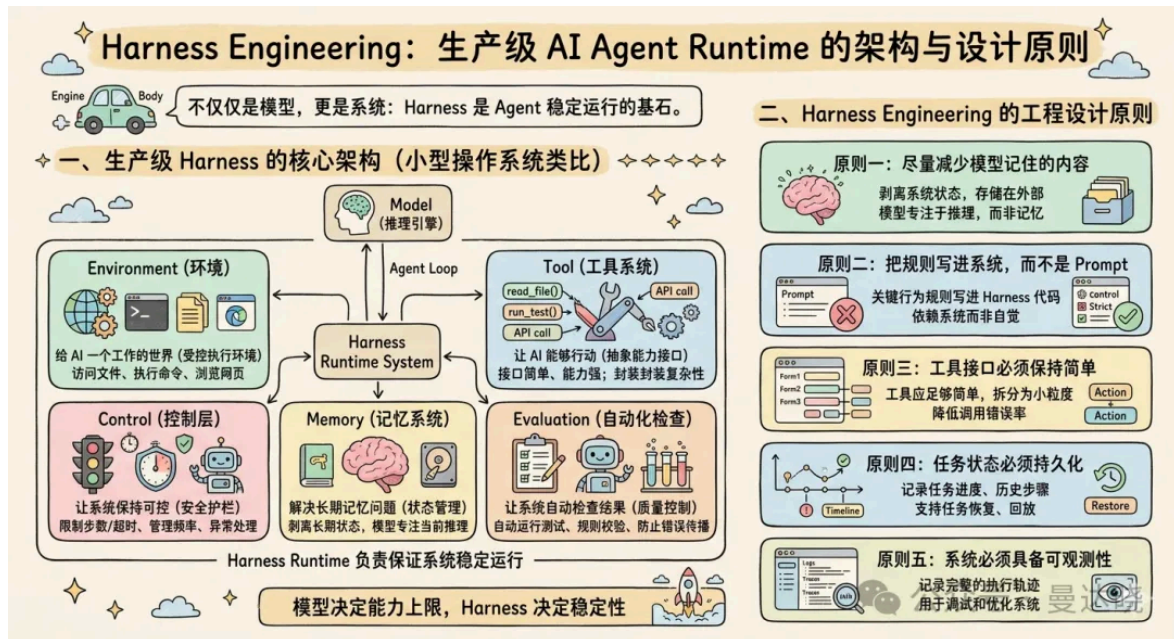


Harness Engineering: 生产级 AI Agent Runtime 的架构与设计原则

原创 AOA1 曼达晓 2026年3月15日 12:04 山东



很多团队第一次做 AI Agent 时, 几乎都会经历同一个阶段: Demo 很惊艳, 上线一周就崩; Agent 会无限循环; 任务做到一半偏离目标; 工具调用乱七八糟; 有时候甚至跑着跑着就“失忆”, 完全忘记自己之前在干什么。

很多人的第一反应是: 模型不够强。但当越来越多团队真正进入 Agent 工程之后, 会慢慢意识到一个事实: 大多数问题, 其实并不在模型, 而在系统。更准确地说, 在 Harness。

如果把 AI Agent 看成一个持续执行任务的系统: 模型负责推理, Agent Loop 负责执行循环, 而 Harness 才是整个系统真正的运行环境。用一个简单类比来说: 模型更像引擎, Agent Loop 更像传动系统, 而 Harness 才是整辆车的结构。

理解这一点之后, 一个更关键的问题就出现了: 如果 Harness 才是决定系统稳定性的关键, 那么一套真正能在生产环境稳定运行的 Harness, 到底应该如何设计?

这正是本文想回答的问题。

在前两篇文章中, 我已经介绍了 Harness 的基本概念, 并分析了 AI Agent 想要稳定执行任务所需要具备的核心能力。本文是这个系列的第三篇, 我会继续往工程层深入, 讨论生产级 Harness 的系统结构与设计原则。

前两篇文章直达链接:

为什么 AI Agent 的关键不是模型, 而是「Harness」

从 Prompt 到 Harness: 2026 AI 竞争决胜点揭秘, 拆解让 AI 稳定干活的 5 大核心能力

一、Harness Engineering: AI Agent 的运行系统工程

在传统软件工程中，开发者主要关注业务逻辑。但在 AI Agent 系统中，工程重心正在发生明显变化。

一个完整的 Agent 系统通常可以被理解为三个层次：

第一层是 **Model Engineering**，负责模型能力，例如推理、规划和生成。

第二层是 **Agent Design**，负责任务策略，例如如何分解任务、如何选择工具以及如何组织执行流程。

第三层则是 **Harness Engineering**，负责构建运行系统，让 Agent 可以稳定地执行长流程任务。

换句话说：**Prompt Engineering** 解决的是模型如何思考，而 **Harness Engineering** 解决的是系统如何运行。

从工程角度来看，**Harness Engineering** 本质上是在设计 **AI Agent 的运行系统 (Agent Runtime System)**。它负责管理环境、工具、执行流程、状态以及结果验证，让 Agent 可以像一个真正的软件系统一样持续运行。

当任务开始变复杂时，这一层往往比模型本身更加关键。

二、一个最小的 Harness，其实非常简单

如果把 Agent 系统极度简化，其核心其实只是一个循环。大多数 Agent 方法都可以被抽象为一个非常简单的执行流程：观察环境、进行推理、选择行动、执行工具，然后根据新的信息继续推理。

用伪代码表示，大致是这样：

```
while not finished:
    observation = environment()
    thought = model(observation)
    action = choose(thought)
    result = run_tool(action)
    update_state(result)
```

这个循环通常被称为 **Agent Loop**。

在很多 Demo 中，这样的结构已经可以完成一些任务，例如自动搜索资料、整理信息、调用 API 或生成代码。看起来，AI 已经开始像一个“会思考的程序”。

但当系统进入真实环境之后，很快就会发现，仅仅依靠这样一个简单循环，是远远不够的。

三、为什么最小 Harness 在现实中很难跑稳

当任务开始变复杂时，Agent 系统会迅速遇到一系列工程问题。

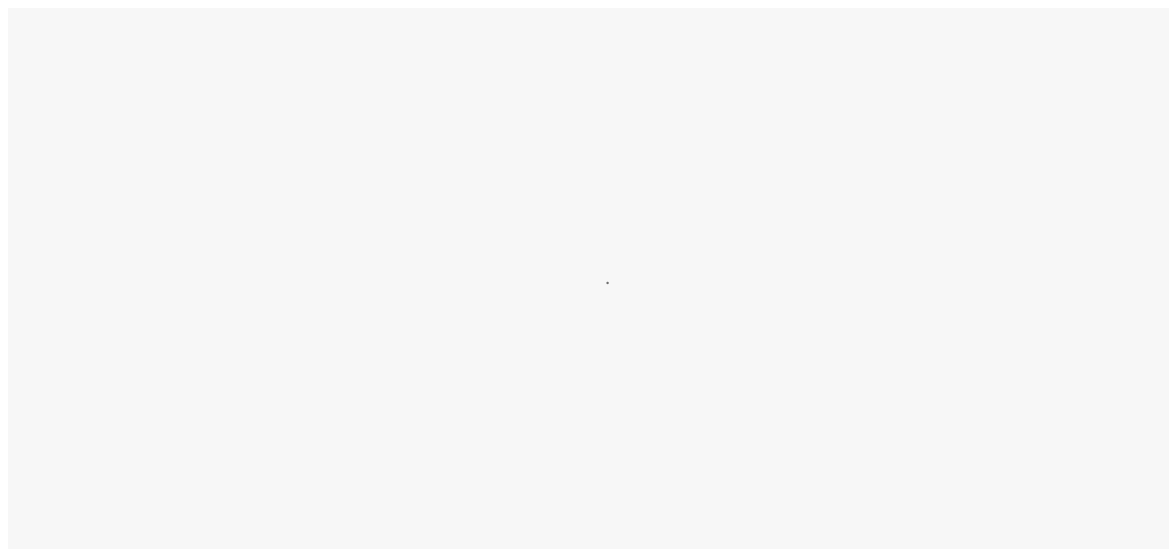
第一个问题是 **上下文膨胀**。每执行一步，系统都会产生新的信息。如果一个任务需要执行几十步甚至上百步，上下文会迅速变得非常庞大。模型不仅推理成本显著上升，还可能因为信息过载而出现推理混乱。

第二个问题是 **工具调用稳定性**。模型虽然可以理解工具描述，但在实际执行中，经常会出现参数错误、工具选择错误或者重复执行等情况。

第三个问题是 **任务漂移**。在长流程任务中，Agent 很容易逐渐偏离最初目标，最终执行一系列与任务无关的操作。

第四个问题是 **状态丢失**。模型本身没有真正的长期记忆，一旦上下文被截断，之前的任务信息就可能消失。

最后是 **结果可靠性问题**。如果系统没有任何验证机制，一次错误推理就可能导致整个任务失败。



这些问题看起来各不相同，但本质上都属于同一个领域：**系统工程问题**。

换句话说，Agent 并不是一个简单的模型调用，而是一个需要完整运行系统支持的软件系统。而这个系统，就是 Harness。

四、生产级 Harness 的核心架构

如果从系统工程的角度来看，一个完整的 AI Agent Harness 其实非常像一个小型操作系统。模型负责推理，但真正让系统稳定运行的，是围绕模型构建的一整套运行环境。

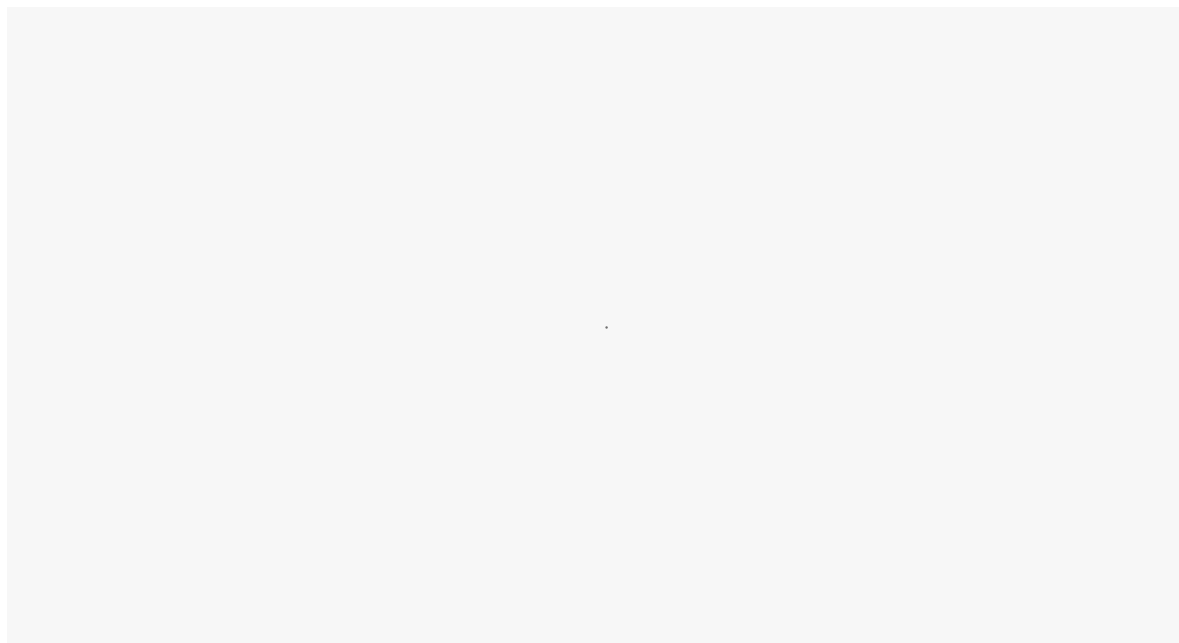
在实践中，一套成熟的 Harness 通常可以抽象为五个核心模块：**Environment、Tool、Control、Memory 和 Evaluation**。这五个模块共同构成了 Agent 的运行系统。模型只负责推理，而 Harness 负责保证整个系统能够持续、稳定地运行。

从整体结构上看，一个典型的 Agent Runtime 大致可以表示为：



在这个结构中，模型不再是整个系统，而只是运行在 Harness Runtime 之上的 **推理引擎**。

下面展开讲下生产级 Harness Architecture 的五层结构，理解了它，就理解了 **AI Agent 系统的真实架构**。



1. Environment: 给 AI 一个可以工作的世界

模型本身其实是完全虚拟的。它既不能访问文件系统，也无法浏览网页，更不能直接操作数据库或执行代码。如果没有额外的系统环境，AI 实际上什么都做不了。

因此 Harness 的第一层，往往是 **Environment 层**。这一层的任务很简单：为 AI 提供一个可以工作的世界。

在工程实践中，这通常意味着**构建一套受控的执行环境**。例如在一个 AI 编程 Agent 中，环境可能包括本地代码仓库、文件系统接口、命令行终端以及运行测试的执行环境。模型可以通过系统提供的接口查看代码、修改文件、运行测试或者执行命令。

从系统视角来看，Environment 层本质上是在解决一个问题：**如何让 AI 与真实世界交互**。

没有这一层，AI 的能力只能停留在文本生成；而一旦有了环境层，AI 才真正具备“执行任务”的可能。

2. Tool: 让 AI 能够行动

即使有了环境，AI 仍然不知道应该如何操作这些资源。文件系统、API 或数据库对于模型来说仍然是不可直接访问的。因此 Harness 需要提供一层更加抽象的能力接口，这就是 **Tool 层**。

Tool 层通常会**把复杂的系统能力封装成一组简单、清晰的工具函数**，例如读取文件、搜索代码、执行测试或调用某个 API。模型在推理过程中不会直接操作底层系统，而是通过这些工具与环境交互。

在很多 Agent 系统中，工具的设计质量往往直接决定了系统能力的上限。如果工具接口过于复杂，模型很容易产生调用错误；如果工具能力过于有限，Agent 的任务范围又会被严重限制。因此成熟的 Harness 通常会遵循一个简单原则：**工具接口要足够简单，但能力要足够强**。

很多人第一次做 Agent 时会误以为系统能力主要来自模型，但在实际工程中会逐渐发现，Agent 能完成什么任务，往往取决于它拥有什么工具。

3. Control: 让系统保持可控

当 Agent 开始执行多步骤任务时，一个新的问题很快就会出现：系统可能会失控。

例如模型可能不断重复某个工具调用，或者在某个步骤中陷入循环；有时它会持续执行无意义操作，甚至偏离原始任务目标。这些问题在 Demo 阶段可能还不明显，但在生产环境中会迅速放大。

因此成熟的 Harness 一定会设计一层 **Control 层**，用于**管理整个 Agent 的执行流程**。

Control 层通常会**负责一些非常关键的运行规则**，例如限制最大执行步数、控制任务超时时间、管理工具调用频率以及处理异常情况。当系统检测到异常状态时，Control 层可以中断任务、重试步骤或者回滚执行状态。

从工程角度来看，Control 层的核心作用其实是为 AI 系统增加一层“安全护栏”。模型仍然可以自由推理，但系统会确保整个执行过程始终处在可控范围之内。

4. Memory: 解决 AI 的长期记忆问题

另一个在实际工程中很快出现的问题是 **记忆能力不足**。模型本身只有上下文记忆，也就是说所有信息都必须放在 Prompt 中。一旦上下文长度达到上限，系统就不得不截断历史信息。

在短任务中这通常不是问题，但在长流程任务中，这种限制会变得非常明显。Agent 可能在执行几十步之后逐渐忘记任务目标，或者无法回忆之前做过的决策。

为了解决这个问题，成熟的 Harness 通常会设计 **Memory 层**。这一层负责把 **任务状态存储在系统中，而不是完全依赖模型上下文。**

例如系统可以记录任务目标、历史步骤、中间结果以及关键决策信息。当 Agent 需要这些信息时，系统可以把相关内容重新注入上下文，而不是简单地保留全部历史记录。

这种结构本质上把 Agent 的记忆分成两部分：上下文承担短期推理所需的信息，而外部存储则承担长期状态管理。通过这种方式，Agent 才能稳定地执行长流程任务。

5. Evaluation: 让系统自动检查结果

即使解决了环境、工具、控制和记忆问题，Agent 系统仍然会面临一个关键挑战：输出质量的不稳定。

模型在推理过程中并不会自动验证自己的结果，因此一旦某一步产生错误，后续步骤就可能在错误基础上继续执行。这在复杂任务中非常常见。

为了解决这一问题，越来越多的 Agent 系统开始在 Harness 中加入 **Evaluation 层**。这一层的任务是 **对关键步骤进行自动验证，而不是直接接受模型输出。**

例如在 AI 编程场景中，系统可以在代码生成后自动运行测试；如果测试失败，就把错误信息返回给模型，让它继续修复代码。在数据处理任务中，系统也可以通过规则校验或额外模型评审来检查结果。

Evaluation 层的存在，本质上是在 **系统中加入一个持续运行的质量控制机制**。它不会完全消除错误，但可以显著降低错误传播的概率。

6. 五层结构的整体作用

当 Environment、Tool、Control、Memory 和 Evaluation 这五个模块组合在一起时，一个完整的 Agent 运行系统就形成了。

Environment 提供可操作的世界，Tool 提供行动能力，Control 负责管理执行流程，Memory 负责长期状态管理，而 Evaluation 则负责验证结果质量。

模型仍然是系统的核心推理引擎，但真正保证系统稳定运行的，是这整套 Harness 结构。

换句话说，Agent 的**能力来自模型**，而 Agent 的**稳定性来自 Harness**。

五、Harness 在真实系统中的形态

在真实工程系统中，这些结构其实已经广泛存在。

以代码 Agent 为例，一个典型系统通常包含：

- Environment 提供本地代码仓库和执行环境；
- Tool 系统封装 read_file、write_file、run_test 等能力；
- Control 系统限制执行步数和任务时间；
- Memory 系统保存任务状态和历史修改记录；
- Evaluation 系统自动运行单元测试。

通过这种结构，Agent 才能够稳定地完成复杂的软件工程任务。

六、Harness Engineering 的设计原则

理解了 Harness 的架构之后，更重要的问题其实是：在工程实践中，如何把这套结构真正做**稳定**。

很多团队在实现 Agent 系统时，往往很快就能把 Environment、Tool、Memory 这些模块搭起来。但系统一旦开始执行复杂任务，很快就会暴露出各种稳定性问题。

原因其实很简单：Agent 系统不仅仅是一个模型调用框架，它更像一个长期运行的软件系统。要让这样的系统稳定运行，仅仅有架构还不够，还需要一整套**工程设计原则**。

在大量 Agent 实践中，逐渐形成了一些非常关键的经验。

第一条原则：尽量减少模型需要记住的内容

很多团队在最初设计 Agent 时，会把大量信息直接放进 Prompt。例如任务历史、执行状态、工具返回结果等全部保存在上下文中。

这种方式在 Demo 阶段通常可以工作，但随着任务复杂度上升，很快就会出现问題。上下文会迅速膨胀，不仅推理成本急剧上升，而且模型在大量历史信息中也更容易出现判断混乱。

成熟的 Harness 通常会采用另一种方式：**把系统状态从模型中剥离出来。**

换句话说，模型只负责当前步骤的推理，而任务状态则存储在系统中。例如数据库、文件系统或者专门的状态管理模块。只有当模型需要相关信息时，系统才会把必要的上下文重新注入到 Prompt 中。

这种设计本质上是在做一件事情：**让模型专注于推理，而不是记忆。**

一旦系统状态管理从 Prompt 中独立出来，Agent 的稳定性通常会有明显提升。

第二条原则：把规则写进系统，而不是写进 Prompt

另一个常见问题是，很多团队习惯在 Prompt 中写大量行为规则，例如要求模型必须遵守某些编码规范，或者要求它必须先完成某个步骤再执行下一步。

从表面上看，这似乎可以解决问题，但在实际运行中，这种方法并不可靠。模型可以理解这些规则，但并不会始终严格遵守。

因此成熟的 Harness 通常会把关键规则写进系统，而不是依赖 Prompt。

例如，如果希望代码必须通过测试，那么最可靠的方式不是在 Prompt 中提醒模型，而是在系统中自动运行测试；如果希望限制某些危险操作，也可以在工具层直接做权限控制。

当规则被写进系统时，模型就不再需要记住这些规则，系统本身就会保证它们被执行。

从工程角度来看，这其实是一种非常经典的设计思想：**让系统保证规则，而不是依赖使用者自觉遵守规则。**

第三条原则：**工具接口必须保持简单**

在很多 Agent 系统中，工具设计是最容易被忽视的一环。开发者往往会把大量复杂功能直接暴露给模型，例如带有多个参数的大型 API。

但在实践中会发现，工具接口越复杂，模型就越容易产生调用错误。参数顺序、字段名称或者调用时机稍微复杂一些，模型就可能理解错误。

因此成熟 Harness 的一个重要经验是：**工具接口必须保持简单。**

很多成功的 Agent 系统都会把复杂操作拆成多个小工具，每个工具只完成一个非常明确的任务。例如读取文件、修改文件或者执行测试，而不是提供一个包含十几个参数的“超级工具”。

当工具接口变得简单清晰时，模型调用工具的稳定性通常会显著提高。

第四条原则：**任务状态必须持久化**

Agent 系统通常需要执行长流程任务，例如代码修改、项目分析或者复杂数据处理。这类任务往往包含多个步骤，甚至可能持续数十次模型推理。

如果任务状态只存在于上下文中，一旦系统重启或者上下文被截断，整个任务状态就会丢失。

因此成熟 Harness 几乎都会设计一套**任务状态系统**，用于持久化存储任务进度。例如记录当前任务目标、已经完成的步骤以及中间结果。当 Agent 继续执行任务时，系统可以重新加载这些信息。

这种设计不仅可以提高系统稳定性，还可以支持更多高级能力，例如任务恢复、任务回放或者执行日志分析。

第五条原则：**系统必须具备可观测性**

在传统软件工程中，可观测性一直是系统稳定运行的重要前提。Agent 系统同样如此。

如果系统没有记录推理过程和执行日志，开发者几乎无法理解 Agent 为什么会做出某个决策。一旦任务执行失败，也很难定位问题究竟出现在 Prompt、工具调用还是状态管理。

因此成熟的 Harness 通常会**记录完整的执行轨迹**，包括**每一步模型推理、工具调用以及系统状态变化**。这些信息不仅可以帮助开发者调试系统，还可以为后续优化提供非常宝贵的数据。

从工程角度来看，可观测性实际上是在给 Agent 系统建立一套“黑匣子”。即使系统出现异常，也能够通过日志还原整个执行过程。

工程经验总结

当这些工程原则组合在一起时，Harness 就不仅仅是一个模型调用框架，而是一套真正能够长期稳定运行的系统。

模型负责推理能力，而 Harness 负责系统稳定性。前者决定系统能做什么任务，而后者决定系统能否持续可靠地完成这些任务。

这也是为什么越来越多团队在实践 Agent 系统之后，会逐渐把更多精力投入到 Harness 设计上。

最后

过去十几年，软件工程的复杂度主要集中在业务逻辑。但在 AI 系统中，复杂度正在发生明显转移。

很多团队在实践 Agent 系统后会发现：真正花时间最多的部分，往往不是模型，而是：

- 工具系统设计
- 执行流程管理
- 上下文控制
- 任务状态管理
- 结果验证机制

这些几乎全部属于 Harness 层。

过去的软件结构通常是：90% 业务逻辑、10% 模型调用。

而在很多 AI 产品中，工程结构正在逐渐变成：模型调用很少、Harness 工程最多。

某种意义上说，软件工程正在出现一种新的分支：**Harness Engineering**。

👍 喜欢作者

Harness · 目录 ≡

<| 上一篇

为什么 AI Agent 的关键不是模型，而是「Harness」

下一篇 >

Tool Router：Harness 如何通过三层架构驯服 AI Agent 的工具爆炸

